

Redes Neuronales

U2 Redes simples

Héctor Maravillo

Section 1

Kesler's Construction

The Perceptron Algorithm (Theodoridis version)

In this formulation, it used the **extended** $(l + 1)$ -**dimensional vectors**

$$x \leftarrow [x^T, 1]^T$$

$$w \leftarrow [w^T, b]^T$$

Let y_i the **target** value of the instance $i = 1, \dots, N$.

The set Y is the subset of the training vectors, which are **misclassified** by the hyperplane defined by the weight vector w .

The variable δ_x is chosen so that

$$\delta_x = \begin{cases} -1 & \text{if } w^T x > 0 \\ +1 & \text{if } w^T x < 0 \end{cases}$$

The Perceptron Algorithm (Theodoridis version)

- Choose $w(0)$ randomly
- Choose α_0
- $t = 0$
- Repeat
 - $Y = \emptyset$
 - For $i = 1$ to n
 - If $\delta_{x_i} y_i \geq 0$ then:

$$Y = Y \cup \{x_i\}$$

- **Perceptron rule**

$$w(t+1) = w(t) - \alpha_k \sum_{x \in Y} \delta_x x$$

- $t = t + 1$
 - Adjust α_k
- Until $Y = \emptyset$

Kesler's Construction

The **generalization of the Perceptron Algorithm** from two-class to M -class task is straightforward.

A **linear discriminant function** w_i , $i = 1, 2, \dots, M$ is defined for each of the classes.

A feature vector x , in the $(I + 1)$ -dimensional space to account for the bias, is classified in class ω_i if

$$w_i^T x > w_j^T x, \quad \forall j \neq i$$

This condition leads to the so-called **Kesler's construction**

Kesler's Construction

For each of the training vectors from class ω_i , $i = 1, 2, \dots, M$, we construct $M - 1$ vectors

$$x_{ij} = [0^T, 0^T, \dots, x^T, \dots, -x^T, \dots, 0^T]^T$$

of dimension $(l + 1)M \times 1$.

That is, they are blocks vectors having zeros everywhere except at the i th and the j th blocks positions, where they have x and $-x$, respectively, for $j \neq i$.

We also construct the block vector

$$w = [w_1^T, \dots, w_M^T]^T$$

Kesler's Construction

If $x \in \omega_i$, this imposes the requirement that

$$w^T x_{ij} > 0, \quad \forall j = 1, 2, \dots, M, j \neq i$$

The task now is to design a linear classifier, in the extended $(I + 1)M$ -dimensional space, so that each of the $(M - 1)N$ training vectors lies in its positive side.

The perceptron algorithm will have no difficulty in solving this problem for us, provided that **such a solution is possible** (that is, if all the training vectors can be correctly classified using a set of linear discriminant functions).

Example

Let us consider a three-class problem in the two-dimensional space. The training vectors for each of the classes are the following:

$$\begin{aligned} \omega_1 : & \quad [1, 1]^T, \quad [2, 2]^T, \quad [2, 1]^T \\ \omega_2 : & \quad [1, -1]^T, \quad [1, -2]^T, \quad [2, -2]^T \\ \omega_3 : & \quad [-1, 1]^T, \quad [-1, 2]^T, \quad [-2, 1]^T \end{aligned}$$

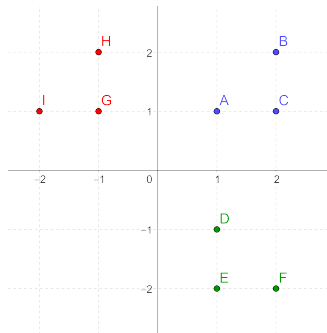


Figure: This is obviously a linearly separable problem, since the vectors of different classes lie in different quadrants.

Example

To compute the linear discriminant functions, we first extend the vectors to the three dimensional space, and then we use **Kesler's construction**. For example

- For $x_1 = [1, 1]^T$ we get

$$\begin{aligned}x_{12} &= [1, 1, 1, -1, -1, -1, 0, 0, 0]^T \\x_{13} &= [1, 1, 1, 0, 0, 0, -1, -1, -1]^T\end{aligned}$$

- For $x_5 = [1, 2]^T$ we get

$$\begin{aligned}x_{51} &= [-1, 2, -1, 1, -2, 1, 0, 0, 0]^T \\x_{53} &= [0, 0, 0, 1, -2, 1, -1, 2, -1]^T\end{aligned}$$

- For $x_9 = [-2, 1]^T$ we get

$$\begin{aligned}x_{91} &= [2, -1, -1, 0, 0, 0, -2, 1, 1]^T \\x_{92} &= [0, 0, 0, 2, -1, -1, -2, 1, 1]^T\end{aligned}$$

Example

Similarly, we obtain the other twelve vectors. To obtain the corresponding weight vectors

$$w_1 = [w_{11}, w_{12}, w_{10}]^T$$

$$w_2 = [w_{21}, w_{22}, w_{20}]^T$$

$$w_3 = [w_{31}, w_{32}, w_{30}]^T$$

we can run the perceptron algorithm by requiring

$$w^T x > 0, \quad w = [w_1^T, w_2^T, w_3^T]^T$$

for each of the eighteen 9-dimensional vectors. That is, we require all the vectors to lie on the same side of the decision hyperplane.

The initial vector of the algorithm $w(0)$ could be choose randomly in the range $(0, 1)$.

Section 2

The Pocket Algorithm

The Pocket Algorithm

A basic requirement for the **convergence** of the perceptron algorithm is the **linear separability** of the classes.

If this is not true, as is usually the case in practice, the perceptron algorithm does not converge.

A variant of the perceptron algorithm was suggested in Gallant (1990)¹, called **the Pocket algorithm** that converges to an optimal solution even if the linear separability condition is not fulfilled.

It can be shown that this algorithm **converges with probability one** to the optimal solution, that is, the one that produces the minimum number of misclassifications.

¹*Perceptron based learning algorithms.*

The Pocket Algorithm

The algorithm is known as the pocket algorithm and consists of the following two steps:

• Step 1

- Initialize the weight vector $w(0)$ randomly.
- Define a stored (in the pocket) vector w_s .
- Set a history counter h_s of the w_s to zero.

• Step 1

- At the t -th iteration step compute the update $w(t+1)$, according to the **perceptron rule**.
- Use the updated weight vector to test the number h of training vectors that are classified correctly.
- If $h > h_s$ replace w_s with $w(t+1)$ and h_s with h .
- Continue the iterations.

References

- Theodoridis, S. & Koutroumbas, K. *Pattern Recognition*, 4th ed., Academic Press, 2009